Python & BDD Skills Assessment Test

CONDENSED — COMBINED EXAM

Sessions: 5
Document Version: 3.0
Updated: September 7, 2026
Assessment Date: _____

Candidate name	
Start time	End time

⚠ WARNING — DO NOT DISTRIBUTE

Ford confidential material. For authorized internal assessment use only. Do not copy, forward, publish, or share outside the approved audience.

FORD CONFIDENTIAL

Table of Contents

Session 1: Python Fundamentals & Virtual Environments 3

Session Overview . . . 3
Scoring Guide - Session 1 (Enhanced) . . . 3
Section 1: Python Fundamentals . . . 3
Section 2: Virtual Environments (VENV) . . . 5

Session 2: Git Basics & Rebase 6

Session Overview . . . 6
Scoring Guide - Session 2 (Enhanced) . . . 6
Section 3: Git Basics . . . 6
Section 4: Git Rebase . . . 7

Session 3: PEP 8 & Ruff 9

Session Overview . . . 9
Scoring Guide - Session 3 (Enhanced) . . . 9
Section 5: PEP 8 & Code Style . . . 9
Section 6: Ruff . . . 10

Session 4: Object-Oriented Programming & Static Analysis 12

Session Overview . . . 12
Scoring Guide - Session 4 (Enhanced) . . . 12
Section 7: Object-Oriented Programming (OOP) . . . 12
Section 8: Static Code Analysis . . . 14

Session 5: Unit Testing & Behavior-Driven Development 16

Session Overview . . . 16
Scoring Guide - Session 5 (Enhanced) . . . 16
Section 9: Unit Testing . . . 16
Completion . . . 18

Assessment Completion Record 19

Session 1: Python Fundamentals & Virtual Environments

Repository: Test-Design-Gherkin-Validation Target Team: Automation Engineers - IVI Testing Team Session Duration: 20 minutes Total Points: 23 Version: 3.0 (Condensed - Enhanced)

Session Overview

This condensed session evaluates Python fundamentals and virtual environment management across three proficiency levels:

Level	Criteria	Expected Performance
Beginner	Basic syntax, simple operations, fundamental concepts	40-60% correct
Intermediate	Applied knowledge, common patterns, practical scenarios	60-80% correct
Advanced	Best practices, optimization, architecture	80-100% correct

Scoring Guide - Session 1 (Enhanced)

Section	Max Points	Time
Python Fundamentals	15	12 minutes
Virtual Environments (VENV)	8	8 minutes
Total	23	20 minutes

Proficiency Levels:

- Beginner: 9-14 points (40-60%)
- Intermediate: 14-18 points (60-80%)
- Advanced: 18-23 points (80-100%)

Section 1: Python Fundamentals

Max Points: 15 | Time: 12 minutes

1.1 Basic Concepts (Beginner - 4 points)

Question 1.1.1 (4 points): Debug and fix this function. It should return a dictionary mapping each sensor name to its average reading, but it has multiple bugs:

```
def calculate_sensor_averages(sensor_data):
    """
    sensor_data: list of tuples like [('temp_sensor', 25.5), ('temp_sensor', 26.0),
        ('speed_sensor', 50)]
    Returns: dict with sensor names as keys and average values
    """
    averages = {}
    for sensor, value in sensor_data:
        if sensor in averages:
            averages[sensor] = (averages[sensor] + value) / 2
        else:
            averages[sensor] = value
    return averages

# Test case (should return {'temp_sensor': 25.75, 'speed_sensor': 50})
data = [('temp_sensor', 25.5), ('temp_sensor', 26.0), ('speed_sensor', 50)]
print(calculate_sensor_averages(data))
```

1.2 Intermediate Concepts (Intermediate - 6 points)

Question 1.2.1 (6 points): Implement a function that processes CAN bus message logs. Given a list of tuples containing (timestamp, message_id, value), return a nested dictionary structure:

```
def parse_can_messages(messages: list[tuple]) -> dict:
    """
    messages: list like [(1.0, 'SPEED', 50), (1.5, 'RPM', 2000), (2.0, 'SPEED', 55)]
    Returns: {
        'SPEED': {'timestamps': [1.0, 2.0], 'values': [50, 55], 'count': 2},
        'RPM': {'timestamps': [1.5], 'values': [2000], 'count': 1}
    }
    Use dictionary comprehension and list comprehension where appropriate.
    """
    # Your code here
    pass
```

1.3 Advanced Concepts (Advanced - 5 points)

Question 1.3.1 (5 points): Analyze this test automation code and explain the problem with the copy operation. Then fix it and explain when you would use shallow vs deep copy in automotive test scenarios:

```
class TestConfiguration:
    def __init__(self, vehicle_params):
        self.params = vehicle_params
        self.test_results = []

test_config_base = {'vehicle': 'Pick-Up', 'ecu_settings': {'timeout': 5, 'retry': 3}}
config1 = TestConfiguration(test_config_base.copy())
config2 = TestConfiguration(test_config_base.copy())

# Tester modifies config2
config2.params['ecu_settings']['timeout'] = 10

# Why does config1 also have timeout=10? How would you fix this?
print(f"Config1 timeout: {config1.params['ecu_settings']['timeout']}")  # Unexpected: 10
```

Section 2: Virtual Environments (VENV)

Max Points: 8 | Time: 8 minutes

2.1 Basic VENV Operations (Beginner - 2 points)

Question 2.1.1 (2 points): You inherited a test project from a colleague. The requirements.txt file shows packages that aren't compatible with Python 3.12. Write the exact commands to:

1. Create a virtual environment with Python 3.10 specifically
2. Activate it
3. Install requirements and verify the Python version being used

---

2.2 VENV Management (Intermediate - 3 points)

Question 2.2.1 (3 points): Your CI/CD pipeline is failing because of conflicting package versions between projects. Explain:

1. How virtual environments solve this problem
2. Why using `pip freeze > requirements.txt` in a project without a venv is problematic
3. Best practices for managing requirements.txt in a multi-developer team

---

2.3 Advanced VENV Scenarios (Advanced - 3 points)

Question 2.3.1 (3 points): Debug this scenario: A developer runs these commands but gets ImportError when trying to import installed packages:

```
python -m venv test_env
pip install pytest
python -m pytest tests/
# ImportError: No module named 'pytest'
```

What's wrong? Provide the corrected sequence and explain two other common virtual environment pitfalls in test automation workflows.

Session 1 of 5 (CONDENSED VERSION) Next Session: Git Basics & Rebase Document Version: 3.0 Last Updated: August 2026 Maintained By: IVI Core Team

Session 2: Git Basics & Rebase

Repository: Test-Design-Gherkin-Validation Target Team: Automation Engineers - IVI Testing Team Session Duration: 20 minutes Total Points: 22 Version: 3.0 (Condensed - Enhanced)

Session Overview

This condensed session evaluates Git fundamentals, branching, merging, and rebasing across three proficiency levels:

Level	Criteria	Expected Performance
Beginner	Basic operations, simple commands, fundamental concepts	40-60% correct
Intermediate	Branching strategies, merge vs rebase, practical scenarios	60-80% correct
Advanced	Complex workflows, conflict resolution, history rewriting	80-100% correct

Scoring Guide - Session 2 (Enhanced)

Section	Max Points	Time
Git Basics	12	10 minutes
Git Rebase	10	10 minutes
Total	22	20 minutes

Proficiency Levels:

- Beginner: 9-13 points (40-60%)
- Intermediate: 13-18 points (60-80%)
- Advanced: 18-22 points (80-100%)

Section 3: Git Basics

Max Points: 12 | Time: 10 minutes

3.1 Fundamental Git Operations (Beginner - 4 points)

Question 3.1.1 (4 points): Debug this git workflow scenario:

```
# Developer's commands:
git add tests/test_ignition.py
git commit -m "Add ignition tests"
git push origin feature/ignition-tests

# Error: src: repository does not exist
# fatal: Could not read from remote repository
```

You check `git remote -v` and see:

```
origin  https://github.com/ford-innersource/wrong-repo.git (fetch)
origin  https://github.com/ford-innersource/wrong-repo.git (push)
```

The correct repository is `ivi-test-automation`. Write the commands to:

1. Fix the remote URL
2. Verify the change
3. Successfully push the changes

3.2 Branching and Merging (Intermediate - 4 points)

Question 3.2.1 (4 points): Analyze this situation:

```
git checkout -b feature/audio-tests
# ... make changes and commit ...
git push origin feature/audio-tests

# Error: fatal: The current branch feature/audio-tests has no upstream branch.
```

After running `git branch -vv`, you see:

```
* feature/audio-tests  a1b2c3d [origin/feature/audio-tests: gone] Add audio tests
  main                 f4e5d6c [origin/main] Latest main
```

1. What does "gone" mean?
2. Write the command to properly set up tracking
3. What's the difference between `git push -u origin feature/audio-tests` and `git push --set-upstream origin feature/audio-tests`?

3.3 Advanced Git Workflows (Advanced - 4 points)

Question 3.3.1 (4 points): Crisis scenario - You committed and pushed production API keys:

```
Commit history:
c3d4e5f (HEAD -> main, origin/main) Add database config  # Contains secrets!
b2c3d4e Update test framework
a1b2c3d Initial commit
```

The keys are in `config/database.yml`. Provide:

1. Commands to remove the file from git history (use git filter-branch or BFG)
2. Steps to ensure the file isn't tracked but stays locally
3. How to prevent this: write the .gitignore pattern AND the pre-commit hook approach
4. Why is simply deleting the file in a new commit insufficient?

Section 4: Git Rebase

Max Points: 10 | Time: 10 minutes

4.1 Rebase Fundamentals (Beginner - 3 points)

Question 4.1.1 (3 points): Analyze these two scenarios and explain the outcome:

Scenario A (merge):

```
git checkout feature/clima
git merge main
```

Scenario B (rebase):

```
git checkout feature/clima
git rebase main
```

Your feature branch has commits A, B, C. Main has commits D, E. Draw the commit graph for each scenario and explain:

1. Which approach creates a merge commit?
2. Which rewrites history?
3. When would you use each in a team environment?

4.2 Interactive Rebase (Intermediate - 3 points)

Question 4.2.1 (3 points): During rebase, you get this conflict:

```
Auto-merging tests/test_vehicle.py
CONFLICT (content): Merge conflict in tests/test_vehicle.py
Could not apply a3b4c5d... Update vehicle tests
```

After opening the file, you see:

```
<<<<<<< HEAD
def test_speed_limit():
    assert vehicle.max_speed == 120
=======
def test_speed_limit():
    assert vehicle.get_max_speed() == 130
>>>>>>> a3b4c5d (Update vehicle tests)
```

Provide:

1. The corrected code (assuming both changes are needed: method name change AND new value)
2. The exact git commands to continue the rebase
3. What would happen if you used `git rebase --skip` instead?

4.3 Advanced Rebase Scenarios (Advanced - 4 points)

Question 4.3.1 (4 points): You need to clean up this messy commit history before code review:

```
a1b2c3d (HEAD -> feature/ivi-tests) WIP - fix typo
b3c4d5e Update tests again
c5d6e7f Fix formatting
d7e8f9g Add IVI integration tests
e9f0g1h Update test config
f1g2h3i Initial IVI test structure
```

Write the interactive rebase commands to:

1. Start interactive rebase for the last 6 commits
2. Squash commits a1b2c3d, b3c4d5e, and c5d6e7f into d7e8f9g
3. Reword commit e9f0g1h to "Configure IVI test environment"
4. Keep f1g2h3i as-is
5. After rebasing, your colleague pushed new changes to main. Write the commands to safely force-push your cleaned history

Bonus (+1 point): Explain why `git push --force` is dangerous and what safer alternative should be used.

Session 2 of 5 (CONDENSED VERSION) Previous Session: Python & VENV Next Session: PEP 8 & Ruff Document Version: 3.0 Last Updated: August 2026 Maintained By: IVI Core Team

Session 3: PEP 8 & Ruff

Repository: Test-Design-Gherkin-Validation Target Team: Automation Engineers - IVI Testing Team Session Duration: 18 minutes Total Points: 18 Version: 3.0 (Condensed - Enhanced)

Session Overview

This condensed session evaluates code style standards and modern linting/formatting tools across three proficiency levels:

Level	Criteria	Expected Performance
Beginner	Basic PEP 8 rules, naming conventions, line length	40-60% correct
Intermediate	Applied style, configuration, practical refactoring	60-80% correct
Advanced	Advanced patterns, tool integration, exceptions to rules	80-100% correct

Scoring Guide - Session 3 (Enhanced)

Section	Max Points	Time
PEP 8 & Code Style	10	10 minutes
Ruff	8	8 minutes
Total	18	18 minutes

Proficiency Levels:

- Beginner: 7-11 points (40-60%)
- Intermediate: 11-14 points (60-80%)
- Advanced: 14-18 points (80-100%)

Section 5: PEP 8 & Code Style

Max Points: 10 | Time: 10 minutes

5.1 Basic PEP 8 Rules (Beginner - 3 points)

Question 5.1.1 (3 points): This code has 8+ PEP 8 violations. Identify ALL of them:

```
import sys,os
from typing import *

def Calculate_VehicleSpeed( distance,time,unit = "mph" ):
    """Calculate speed"""
    Speed=distance/time;result= Speed if unit=="mph" else Speed*1.60934
    return result

x=Calculate_VehicleSpeed(100,2)
print( "Speed: %s"%x )
```

5.2 PEP 8 in Practice (Intermediate - 4 points)

Question 5.2.1 (4 points): Refactor this automotive test helper function to follow ALL PEP 8 guidelines. Maintain functionality:

```
def check_ecu_response(ecu_id,expected_codes,timeout,retry_count=3,log_failures=True):
    """Check ECU response"""
    for i in range(retry_count):
        response=get_ecu_data(ecu_id,timeout)
        if response['status_code'] in expected_codes:return True
        else:
            if log_failures==True:print(f"Attempt {i+1} failed: got {response['status_code']}")
    return False

result=check_ecu_response('ECU_001',[200,201,204],5.0,retry_count=5,log_failures=False) if
    is_prod() else check_ecu_response('ECU_001',[200],2.0)
```

5.3 Advanced Style Considerations (Advanced - 3 points)

Question 5.3.1 (3 points): Code review scenario - A developer argues these PEP 8 violations are justified:

```
# Violation 1: Line > 79 characters
expected_can_messages = {'VEHICLE_SPEED': 0x123, 'ENGINE_RPM': 0x456, 'TRANSMISSION_STATE':
    0x789, 'BRAKE_STATUS': 0x ABC, 'STEERING_ANGLE': 0xDEF}

# Violation 2: Variable name
X = transformation_matrix @ input_vector  # Linear algebra operation

# Violation 3: Lambda instead of def
validate = lambda x: x > 0 and x < 200 and x % 2 == 0
```

For each:

1. State whether the violation is justified and why/why not
2. Provide the corrected code if not justified
3. Explain the principle behind your decision

Section 6: Ruff

Max Points: 8 | Time: 8 minutes

6.1 Ruff Basics (Beginner - 2 points)

Question 6.1.1 (2 points): After running `ruff check .` on your test suite, you get:

```
tests/test_ivi.py:23:5: F841 Local variable `response` is assigned but never used
tests/test_ivi.py:45:1: E501 Line too long (120 > 88 characters)
tests/helpers.py:12:1: I001 Import block is un-sorted or un-formatted
```

Write the commands to:

1. Auto-fix all fixable issues
2. Show what would be fixed without actually fixing (dry-run)
3. Check only the tests/ directory

6.2 Ruff Configuration (Intermediate - 3 points)

Question 6.2.1 (3 points): You need to configure Ruff for an automotive test project. Create a `pyproject.toml` configuration that:

1. Extends line length to 120 characters
2. Ignores E501 (line too long) and F841 (unused variable) for test files only
3. Enables these rule sets: pyflakes (F), pycodestyle (E, W), isort (I), and pytest style (PT)
4. Excludes the `vendor/` and `build/` directories

6.3 Advanced Ruff Usage (Advanced - 3 points)

Question 6.3.1 (3 points): Debug this CI/CD pipeline issue:

```
# .github/workflows/test.yml
- name: Lint with Ruff
  run: |
    pip install ruff
    ruff check . --fix
    git diff --exit-code  # Fails here!
```

The pipeline fails because Ruff modifies files. The team wants:

1. Ruff to check but NOT auto-fix in CI
2. Developers should auto-fix locally before committing
3. CI should fail with clear messages showing what needs fixing

Provide:

1. The corrected CI configuration
2. A pre-commit hook setup so developers auto-fix before committing
3. Explain why `ruff check --select ALL` might be problematic and suggest a better approach

---

Session 3 of 5 (CONDENSED VERSION) Previous Session: Git Basics & Rebase Next Session: OOP & Static Analysis Document Version: 3.0 Last Updated: August 2026 Maintained By: IVI Core Team

Session 4: Object-Oriented Programming & Static Analysis

Repository: Test-Design-Gherkin-Validation Target Team: Automation Engineers - IVI Testing Team Session Duration: 20 minutes Total Points: 23 Version: 3.0 (Condensed - Enhanced)

Session Overview

This condensed session evaluates OOP principles and static code analysis across three proficiency levels:

Level	Criteria	Expected Performance
Beginner	Basic OOP concepts, class structure, simple analysis	40-60% correct
Intermediate	Inheritance, polymorphism, type checking, configuration	60-80% correct
Advanced	SOLID principles, design patterns, advanced analysis	80-100% correct

Scoring Guide - Session 4 (Enhanced)

Section	Max Points	Time
Object-Oriented Programming (OOP)	15	12 minutes
Static Code Analysis	8	8 minutes
Total	23	20 minutes

Proficiency Levels:

- Beginner: 9-14 points (40-60%)
- Intermediate: 14-18 points (60-80%)
- Advanced: 18-23 points (80-100%)

Section 7: Object-Oriented Programming (OOP)

Max Points: 15 | Time: 12 minutes

7.1 OOP Fundamentals (Beginner - 5 points)

Question 7.1.1 (5 points): Debug and complete this ECU simulator class. It has multiple issues:

```
class ECUSimulator:
    def __init__(self, ecu_id, protocol):
        self.ecu_id = ecu_id
        self.protocol = protocol
        self.state = "disconnected"
        self.message_count = 0

    def connect():
        state = "connected"
        print(f"ECU {self.ecu_id} connected via {protocol}")

    def send_message(self, message):
        if self.state != "connected":
            return "Error: Not connected"
        message_count += 1
        return f"Sent: {message}"

    def get_stats(self):
        return f"{self.ecu_id}: {message_count} messages sent"

# Test code (should work after fixes):
ecu = ECUSimulator("ECU_01", "CAN")
ecu.connect()
print(ecu.send_message("PING"))
print(ecu.get_stats())  # Should show: BCM_01: 1 messages sent
```

List all bugs and provide the corrected class.

7.2 Inheritance and Polymorphism (Intermediate - 5 points)

Question 7.2.1 (5 points): Design a test framework class hierarchy:

Base class `TestFramework` with:

- `setup()`, `teardown()`, `execute(test_case)` methods
- `results` list to store test results

Two derived classes:

- `BDDFramework`: Override `execute()` to parse Gherkin scenarios
- `APIFramework`: Override `execute()` to handle REST API tests

Implement the classes demonstrating:

1. Proper method overriding
2. Calling parent class methods using `super()`
3. Polymorphic behavior (same interface, different implementations)

Provide code and explain why Python doesn't have traditional method overloading.

7.3 Advanced OOP Concepts (Advanced - 5 points)

Question 7.3.1 (5 points): Refactor this code to follow SOLID principles:

```
class VehicleTestManager:
    def __init__(self, vehicle_type):
        self.vehicle_type = vehicle_type
        self.test_results = []

    def run_test(self, test_name):
        if self.vehicle_type == "EV":
            # Run EV-specific tests
            battery_level = self.check_battery()
            charging_status = self.check_charging()
            self.test_results.append({"test": test_name, "battery": battery_level})
        elif self.vehicle_type == "ICE":
            # Run ICE-specific tests
            fuel_level = self.check_fuel()
            engine_temp = self.check_engine_temp()
            self.test_results.append({"test": test_name, "fuel": fuel_level})

        # Save to database
        self.save_to_db(self.test_results)
        # Send email notification
        self.send_email(f"Test {test_name} completed")

    def check_battery(self): return 80
    def check_charging(self): return "charging"
    def check_fuel(self): return 45
    def check_engine_temp(self): return 95
    def save_to_db(self, data): print("Saving to DB")
    def send_email(self, msg): print(f"Email: {msg}")
```

Identify which SOLID principles are violated and provide refactored code following:

- Single Responsibility Principle (SRP)
- Open/Closed Principle (OCP)
- Dependency Inversion Principle (DIP)

Section 8: Static Code Analysis

Max Points: 8 | Time: 8 minutes

8.1 Static Analysis Basics (Beginner - 2 points)

Question 8.1.1 (2 points): After running `mypy` on your test code, you get these errors:

```
tests/test_vehicle.py:15: error: Argument 1 to "get_speed" has incompatible type "str";
    expected "int"
tests/test_vehicle.py:23: error: Incompatible return value type (got "None", expected
    "List[str]")
tests/helpers.py:8: error: Name "responce" is not defined
```

Which of these can static analysis detect BEFORE runtime? Explain why static analysis can't catch all bugs.

8.2 Type Hints and mypy (Intermediate - 3 points)

Question 8.2.1 (3 points): Add proper type hints to this function and fix any type safety issues:

```
def process_test_results(results, filter_status=None, limit=10):
    """Process and filter test results.

    Args:
        results: List of test result dictionaries
        filter_status: Optional status to filter by ('pass', 'fail', 'skip')
        limit: Maximum number of results to return

    Returns:
        Filtered list of test results
    """
    if filter_status:
        results = [r for r in results if r['status'] == filter_status]

    if limit:
        results = results[:limit]

    return results

# How would mypy catch this error?
results = process_test_results("not a list", filter_status=123)
```

8.3 Advanced Analysis (Advanced - 3 points)

Question 8.3.1 (3 points): Analyze this code's complexity and security issues:

```
def validate_and_execute_test(test_config: dict, user_input: str) -> bool:
    # Complexity: Multiple nested conditions
    if test_config.get('enabled'):
        if test_config.get('environment') == 'prod':
            if user_input.startswith('test_'):
                if len(user_input) > 5:
                    if user_input not in test_config.get('blocked', []):
                        # Security: Command injection risk
                        result = eval(f"run_{user_input}()")
                        if result:
                            if result.status == 'success':
                                return True
                            else:
                                return False
    return False
```

Provide:

1. Calculate or estimate the cyclomatic complexity
2. Identify the security vulnerability and explain the risk
3. Refactor to reduce complexity to < 5 and fix security issue
4. What static analysis tools can detect these issues?

Session 4 of 5 (CONDENSED VERSION) Previous Session: PEP 8 & Ruff Next Session: Unit Testing Document Version: 3.0 Last Updated: August 2026 Maintained By: IVI Core Team

Session 5: Unit Testing & Behavior-Driven Development

Repository: Test-Design-Gherkin-Validation Target Team: Automation Engineers - IVI Testing Team Session Duration: 15 minutes Total Points: 14 Version: 3.0 (Condensed - Enhanced)

Session Overview

This condensed session evaluates unit testing practices and BDD/Gherkin knowledge across three proficiency levels:

Level	Criteria	Expected Performance
Beginner	Basic testing, simple scenarios, Gherkin syntax	40-60% correct
Intermediate	Mocking, fixtures, best practices, refactoring	60-80% correct
Advanced	Parametrization, coverage, scenario outlines, patterns	80-100% correct

Scoring Guide - Session 5 (Enhanced)

Section	Max Points	Time
Unit Testing	14	15 minutes
Total	14	15 minutes

Proficiency Levels:

- Beginner: 6-8 points (40-60%)
- Intermediate: 8-11 points (60-80%)
- Advanced: 11-14 points (80-100%)

Section 9: Unit Testing

Max Points: 14 | Time: 15 minutes

9.1 Unit Testing Fundamentals (Beginner - 4 points)

Question 9.1.1 (4 points): Debug this failing test and fix both the test AND the function:

```
import pytest

def calculate_fuel_efficiency(distance_km: float, fuel_liters: float) -> float:
    """Calculate fuel efficiency in km/L"""
    return distance_km / fuel_liters

def test_fuel_efficiency():
    # Test 1: Normal case
    assert calculate_fuel_efficiency(100, 5) == 20.0

    # Test 2: Edge case - what should happen here?
    assert calculate_fuel_efficiency(100, 0) == 0

    # Test 3: Negative values - missing validation
    assert calculate_fuel_efficiency(-50, 5) == -10.0

    # Test 4: String input - should raise TypeError
    calculate_fuel_efficiency("100", "5")
```

Provide:

1. The corrected function with proper error handling
2. Corrected test with appropriate assertions and pytest.raises where needed
3. Explain what each test case validates

9.2 Mocking and Fixtures (Intermediate - 5 points)

Question 9.2.1 (5 points): Create a comprehensive test for this CAN bus interface:

```
class CANInterface:
    def __init__(self, bus_name: str, bitrate: int):
        self.bus = can.interface.Bus(bus_name, bustype='socketcan', bitrate=bitrate)
        self.logger = logging.getLogger(__name__)

    def send_message(self, msg_id: int, data: bytes) -> bool:
        try:
            msg = can.Message(arbitration_id=msg_id, data=data, is_extended_id=False)
            self.bus.send(msg)
            self.logger.info(f"Sent message {msg_id}: {data.hex()}")
            return True
        except can.CanError as e:
            self.logger.error(f"Failed to send: {e}")
            return False
```

Write tests that:

1. Mock the CAN bus (don't require actual hardware)
2. Verify message is sent with correct parameters
3. Test error handling when bus.send() raises CanError
4. Verify logging calls with correct arguments
5. Use pytest fixtures to set up the mock objects

9.3 Advanced Testing Patterns (Advanced - 5 points)

Question 9.3.1 (5 points): Implement a parametrized test suite for state machine testing:

```
class IgnitionStateMachine:
    def __init__(self):
        self.state = 'OFF'
        self.transitions = {
            'OFF': ['ACC', 'ON'],
            'ACC': ['OFF', 'ON'],
            'ON': ['ACC', 'START'],
            'START': ['ON']
        }

    def transition(self, new_state: str) -> bool:
        if new_state in self.transitions.get(self.state, []):
            self.state = new_state
            return True
        return False
```

Create tests using `pytest.mark.parametrize` that:

1. Test all valid state transitions
2. Test all invalid state transitions
3. Test state transition sequences (e.g., OFF -> ACC -> ON -> START)
4. Include edge cases (invalid states, None values)
5. Use fixtures to reset state machine between tests

Bonus (+2 points): Add test coverage measurement and explain how to achieve 100% branch coverage for this class.

Session 5 of 5 (CONDENSED VERSION) Previous Session: OOP & Static Analysis Document Version: 3.0 Last Updated: August 2026 Maintained By: IVI Core Team

---

Completion

Congratulations on completing all 5 condensed sessions of the Python & BDD Skills Assessment!

Next Steps:

1. Submit all 5 answer files to your lead
2. Review your results with your lead
3. Identify knowledge gaps
4. Create a learning plan for improvement
5. Schedule follow-up assessment if needed (for specific sections)

For questions or concerns about this assessment, contact the IVI Core Team.

Assessment Completion Record

Use this page for evaluator notes and the final proficiency determination.

Session	Maximum points	Score
1 — Python Fundamentals & VENV	23	
2 — Git Basics & Rebase	22 (+1 bonus)	
3 — PEP 8 & Ruff	18	
4 — OOP & Static Analysis	23	
5 — Unit Testing	14 (+2 bonus)	
TOTAL	**100 (+3 bonus)**	

Final proficiency: ☐ Beginner ☐ Intermediate ☐ Advanced

Evaluator notes:

Evaluator signature: _____